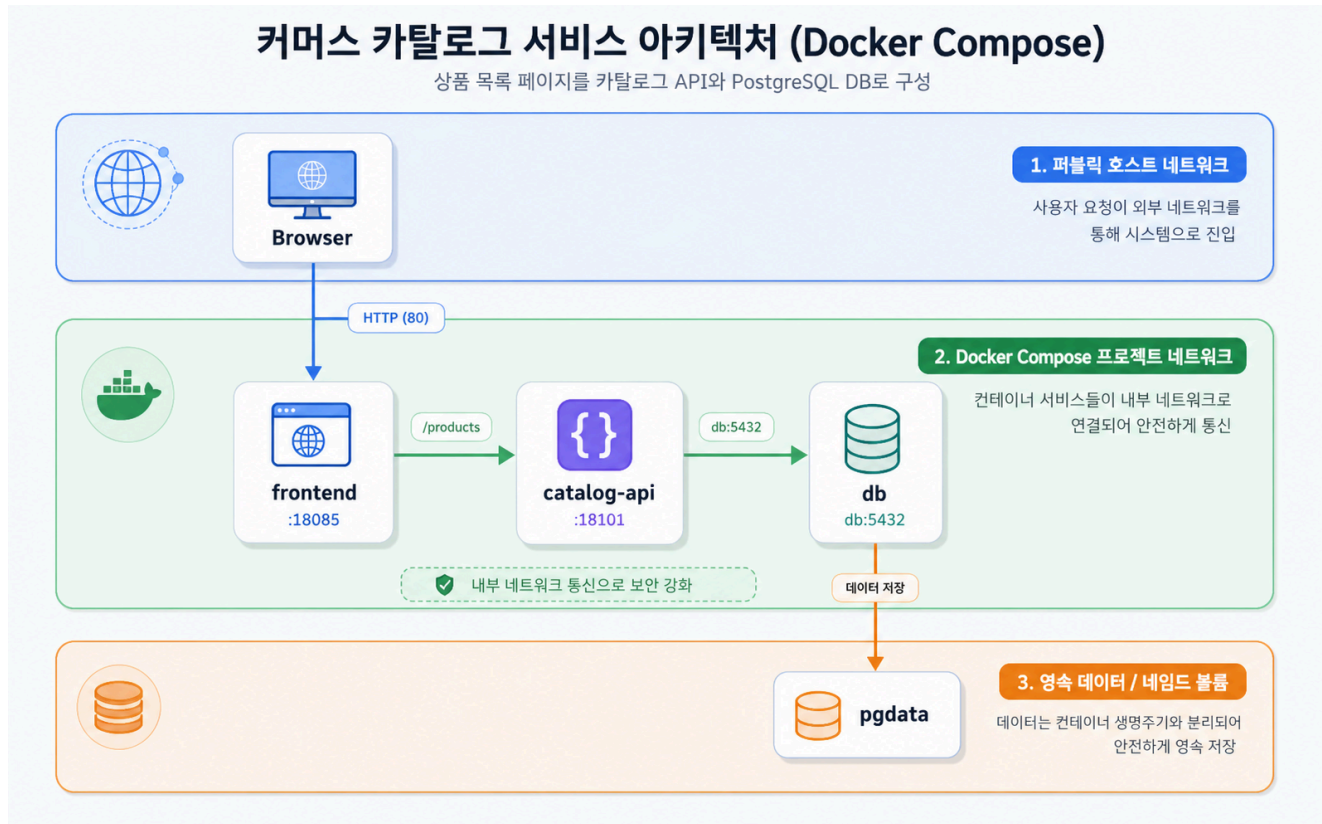


2교시: 쿠팡형 커머스 카탈로그 template



수업 목표

- frontend, catalog API, PostgreSQL을 별도 service로 실행하는 기본 구조를 읽는다.
- host port와 container internal port를 구분한다.
- db-checker 로그로 service name 기반 DB 연결을 확인한다.

언제 쓰는가

커머스 화면은 하나처럼 보이지만 뒤에는 상품 카탈로그 API와 상품 DB가 있다. W1D4의 커머스 아키텍처를 Compose로 줄이면 frontend, catalog API, PostgreSQL 세 덩어리로 시작할 수 있다.

Template

```
cd week2/day5/labs/compose-architectures/01-web-postgres
docker compose config
docker compose up -d
docker compose ps
```

구성:

Service	역할	공개 범위
frontend	nginx static web app	host 18085
catalog-api	products table REST API	host 18101
db	PostgreSQL 16	Compose network 내부
db-checker	DB 연결 확인 app	logs로 결과 확인

Check

```
curl -I http://localhost:18085
curl -s http://localhost:18101/products
docker compose exec db psql -U postgres -d app -c "SELECT current_database();"
docker compose logs db-checker --tail 30
```

Expected:

```
HTTP/1.1 200 OK
"name":"local-market-starter-kit"
products
```

실무 해석

host에서는 localhost:18085로 frontend에 접근하고 localhost:18101/products로 API를 확인한다. 하지만 API container는 DB에 localhost가 아니라 service name db로 붙는다. 이 차이가 host port와 internal service DNS의 핵심이다.

Cleanup

```
docker compose down
# DB를 초기화할 때만
# docker compose down -v
```